

FULL STACK DEVELOPMENT
NEXT.JS

AULA 03

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	14
O QUE VOCÊ VIU NESTA AULA?	18
REFERÊNCIAS.....	19

EMSE

O QUE VEM POR AÍ?

Nesta aula, você aprenderá como funcionam API Routes em aplicações Next, suas vantagens e a diferença em utilizá-las ou não. Além disso, abordaremos as suas boas práticas e como potencializar o desenvolvimento em Next com seu uso. Vamos lá?

EMSE

HANDS ON

Nesta aula, abordamos a utilização e não utilização de API Routes. Vimos como a implementação dessas rotas é simples e como nos traz maior e melhor manutenibilidade do nosso código.

A seguir temos o código da aula:

Page.tsx

```
'use client'

import Link from "next/link";
import { useRouter } from "next/navigation";
import { useEffect, useState } from "react";
import { Bounce, ToastContainer, toast } from "react-toastify";

export default function Home() {
  const router = useRouter()
  const [name, setName] = useState("")
  const [password, setPassword] = useState("")

  const handleLogin = async (e: any) => {
    e.preventDefault();

    try {
      const res = await fetch('/api/login', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
```

```
    },  
    body: JSON.stringify({ name, password }),  
  });  
  
  const data = await res.json();  
  if (!res.ok) {  
    toast.error(`Erro ao registrar usuário`, {  
      position: "bottom-center",  
      autoClose: 5000,  
      hideProgressBar: true,  
      closeOnClick: true,  
      pauseOnHover: true,  
      draggable: true,  
      progress: undefined,  
      theme: "colored",  
      transition: Bounce,  
    });  
    throw new Error(data.message || 'Algo deu errado na autenticação');  
  }  
  router.push('/persons')  
  console.log('Login bem-sucedido:', data);  
} catch (error) {  
}  
}  
;  
return (
```

```

<main className='flex flex-1 flex-col h-screen w-screen justify-center items-
center bg-background-900'>

  <p className='font-sans text-2xl text-fontColor-900'>FIAPCHAT</p>

  <div className='flex flex-1 flex-col max-h-[45vh] w-[80vw] justify-center items-
center bg-background-800'>

    <p className='font-sans text-1xl text-fontColor-900'>Digite um nome de
usuário</p>

    <input

      className='font-sans text-lg text-fontColor-900 bg-background-900 border
border-fontColor-900 rounded-xl p-2 w-5/6'

      type="text"

      placeholder=""

      onChange={(e) => setName(e.target.value)}/>

    <p className='font-sans mt-2 text-1xl text-fontColor-900'>Digite sua
senha</p>

    <input

      className='font-sans text-lg text-fontColor-900 bg-background-900 border
border-fontColor-900 rounded-xl p-2 w-5/6'

      type="password"

      placeholder=""

      onChange={(e) => setPassword(e.target.value)}/>

    <div className="flex flex-1 flex-row mt-2 w-[40%] max-h-12 ml-auto mr-auto
justify-between items-center">

      <Link href="/register" className='w-[max-content] p-2 font-sans bg-
fontColor-900 rounded-xl hover:opacity-80'>Não possui cadastro? Cadastre-se
aqui!</Link>

```

```

    <button disabled={{password == " " || name == ""}}onClick={handleLogin}
    className='w-32 font-sans p-2 bg-fontColor-900 rounded-xl hover:opacity-
    80'>Entrar</button>

    </div>
  </div>

  <ToastContainer
    position="bottom-center"
    autoClose={5000}
    hideProgressBar
    newestOnTop={false}
    closeOnClick
    rtl={false}
    pauseOnFocusLoss
    draggable
    pauseOnHover
    theme="colored"
    transition={Bounce}
  />
</main>

);
}

```

Login.ts

```

// pages/api/login.js

export default async function handler(req, res) {

```

```
if (req.method === 'POST') {  
  try {  
    const { name, password } = req.body;  
    const response = await fetch('http://localhost:3333/api/users/login', {  
      method: 'POST',  
      headers: {  
        'Content-Type': 'application/json',  
      },  
      body: JSON.stringify({  
        name,  
        password,  
      }),  
    });  
  
    const data = await response.json();  
  
    if (response.ok) {  
      return res.status(200).json(data);  
    } else {  
      return res.status(response.status).json(data);  
    }  
  } catch (error) {  
    return res.status(500).json({ message: "Erro ao conectar com a API de terceiros." });  
  }  
} else {
```



```
res.setHeader('Allow', ['POST']);

res.status(405).end(`Método ${req.method} não permitido`);

}

}
```

index.tsx (dentro da pasta register)

```
import Link from "next/link";
import { useRouter } from "next/router";
import { useState } from "react";
import { Bounce, ToastContainer, toast } from "react-toastify";
import 'react-toastify/dist/ReactToastify.css';

export default function Register() {
  const router = useRouter();
  const [name, setName] = useState("")
  const [password, setPassword] = useState("")

  const handleLogin = async (e:any) => {
    e.preventDefault();

    const response = await fetch('http://localhost:3333/api/users/register', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({ name, password }),
    });
  };
}
```

```
if (response.ok) {  
  router.push('/');  
} else {  
  toast.error(`Erro ao registrar usuário`, {  
    position: "bottom-center",  
    autoClose: 5000,  
    hideProgressBar: true,  
    closeOnClick: true,  
    pauseOnHover: true,  
    draggable: true,  
    progress: undefined,  
    theme: "colored",  
    transition: Bounce,  
  });  
}  
};  
return (  
  <main className='flex flex-1 flex-col h-screen w-screen justify-center items-center bg-background-900'>  
    <p className='font-sans text-2xl text-fontColor-900'>CADASTRE-SE</p>  
    <div className='flex flex-1 flex-col max-h-[45vh] w-[80vw] justify-center items-center bg-background-800'>  
      <p className='font-sans text-1xl text-fontColor-900'>Digite um nome de usuário</p>  
      <input
```

```

        className='font-sans text-lg text-fontColor-900 bg-background-900 border
border-fontColor-900 rounded-xl p-2 w-5/6'

        type="text"

        placeholder=""

        onChange={(e) => setName(e.target.value)}>

<p className='font-sans mt-2 text-1xl text-fontColor-900'>Digite sua
senha</p>

<input

        className='font-sans text-lg text-fontColor-900 bg-background-900 border
border-fontColor-900 rounded-xl p-2 w-5/6'

        type="password"

        placeholder=""

        onChange={(e) => setPassword(e.target.value)}>

<div className="flex flex-1 flex-row mt-2 w-[40%] max-h-12 ml-auto mr-auto
justify-between items-center">

        <Link href="/" className='w-32 mt-2 font-sans p-2 text-center bg-
background-900 border-[2px] border-fontColor-900 rounded-xl hover:opacity-
80'>VOLTAR</Link>

        <button disabled={{(password == " || name == ")}} className='w-32 mt-2 font-
sans text-center p-2 bg-fontColor-900 rounded-xl hover:opacity-80'
onClick={handleLogin}>CONFIRMAR</button>

    </div>

</div>

<ToastContainer

    position="bottom-center"

    autoClose={5000}

    hideProgressBar

```

```

    newestOnTop={false}

    closeOnClick

    rtl={false}

    pauseOnFocusLoss

    draggable

    pauseOnHover

    theme="colored"

    transition={Bounce}

  />
</main>
);
}

```

_app.tsx

```

import { AppProps } from 'next/app';
import '../app/globals.css';

function Pages({ Component, pageProps }: AppProps) {
  return <Component {...pageProps} />;
}

export default Pages;

```

Index.tsx (dentro da pasta persons)

```

export default function Persons() {

```

```
return (  
  <main className='flex flex-1 flex-col h-screen w-screen justify-center items-  
center bg-background-900'>  
    <p className='font-sans text-2xl text-fontColor-900'>listar as pessoas</p>  
  </main>  
);  
}
```

Não deixe de praticar!

SAIBA MAIS

O uso de API Routes em Next.js oferece uma forma integrada e eficiente de construir APIs dentro de uma aplicação Next.js, eliminando a necessidade de um servidor back-end separado. Esta funcionalidade permite que você crie funções server-side diretamente no seu projeto Next.js, facilitando o desenvolvimento de aplicações full-stack. Vejamos uma visão geral de como funcionam e como você pode utilizá-las:

Criação de API Routes

Para criar uma API Route em Next.js, você simplesmente adiciona arquivos JavaScript ou TypeScript dentro da pasta `pages/api` do seu projeto. Cada arquivo nessa pasta se torna uma rota de API acessível pela mesma URL que o caminho do arquivo. Por exemplo, um arquivo chamado `pages/api/user.js` é acessível por meio de `/api/user`.

Estrutura de um API Route

Um arquivo de API Route exporta uma função, que pode ser assíncrona, que recebe dois parâmetros: o objeto `req` (request) e o objeto `res` (response). Esses objetos permitem que você manipule a requisição recebida e envie uma resposta ao cliente.

Manipulação de métodos HTTP

Você pode verificar o método HTTP da requisição (GET, POST, PUT, DELETE etc.) usando o objeto `req` para criar lógicas condicionais e responder adequadamente. Isso permite a criação de APIs RESTful diretamente em seu projeto Next.js.

Benefícios das API Routes

- **Simplicidade:** integradas diretamente ao seu projeto Next.js, as API Routes simplificam a arquitetura de sua aplicação, eliminando a necessidade de um back-end separado.
- **Desenvolvimento full-stack:** facilita o desenvolvimento de aplicações full-stack ao permitir que você escreva o código do lado do servidor em um ambiente familiar.

- Desempenho e otimização: aproveita as otimizações de desempenho e as funcionalidades de renderização do lado do servidor do Next.js.
- Flexibilidade: suporta uma ampla gama de casos de uso, desde APIs simples até funcionalidades mais complexas, como manipulação de formulários, autenticação e integração com bancos de dados.

Boas práticas

- Mantenha suas funções API concisas e focadas em uma única responsabilidade.
- Use variáveis de ambiente para armazenar informações sensíveis.
- Implemente tratamento de erros adequado para melhorar a robustez de sua API.

As API Routes do Next.js são uma poderosa ferramenta para desenvolvedores(as) que buscam construir aplicações full-stack eficientes e escaláveis, aproveitando as vantagens do framework Next.js.

Para aprofundar nosso entendimento sobre as API Routes em Next.js, vamos explorar alguns aspectos avançados e dicas práticas que podem ajudar a maximizar a eficiência e a eficácia das suas APIs.

Middleware personalizado

Você pode criar middleware personalizado para suas API Routes para manipular tarefas comuns, como autenticação, logging ou validação de requisições antes de chegarem ao manipulador da rota. Isso permite reutilizar a lógica em várias rotas, mantendo seu código DRY (Don't Repeat Yourself).

Rotas dinâmicas

Assim como nas páginas, você pode criar API Routes dinâmicas utilizando colchetes ([...]) no nome do arquivo para capturar parâmetros da URL. Isso é útil para construir APIs RESTful onde você precisa operar em recursos específicos identificados por IDs ou outros parâmetros.

Manipulação de dados complexos

Para lidar com dados complexos ou operações que exigem um tempo de processamento significativo, você pode usar funções assíncronas nas suas API Routes. Isso é especialmente útil ao realizar consultas a bancos de dados ou chamadas a APIs externas.

Segurança

Ao desenvolver API Routes, a segurança deve ser uma prioridade. Aqui estão algumas dicas para manter suas APIs seguras:

- Validação de entrada: sempre valide as entradas do usuário para proteger contra injeções SQL, XSS, e outros ataques.
- Limitação de taxa: implemente limitação de taxa para evitar ataques de negação de serviço (DoS).
- CORS: configure adequadamente os cabeçalhos CORS para controlar quais origens podem acessar suas APIs.

Performance e otimização

- Cacheamento: para melhorar a performance, considere o cacheamento de respostas, especialmente para dados que não mudam frequentemente.
- Compressão: use compressão para reduzir o tamanho das respostas, acelerando assim o tempo de transferência.

SSR (Server-Side Rendering) e SSG (Static Site Generation)

Uma das grandes vantagens do Next.js é sua capacidade de renderizar páginas no lado do servidor (SSR) ou gerar sites estáticos (SSG). As API Routes complementam essas funcionalidades permitindo que os dados sejam buscados no lado do servidor antes da renderização ou durante a construção do site. Isso significa que você pode construir aplicações completas com Next.js sem a necessidade de um servidor back-end separado.

API Routes e Serverless

Quando você implanta um aplicativo Next.js (por exemplo, na Vercel ou em outras plataformas compatíveis), cada API Route pode ser automaticamente implementada como uma função serverless. Isso significa que cada rota tem sua própria função lambda, que escala automaticamente de acordo com a demanda. Isso torna as API Routes uma solução altamente escalável e eficiente para construir APIs, especialmente para aplicações que experimentam volumes variáveis de tráfego.

Práticas recomendadas para API Routes

- Separação de lógica: mantenha a lógica específica da API separada da lógica de renderização de páginas. Isso não apenas mantém seu código organizado, mas também facilita a manutenção.
- Segurança: implemente autenticação e autorização nas suas API Routes para proteger dados sensíveis. O Next.js não inclui uma solução de autenticação integrada para API Routes, então você precisará implementar a sua própria ou utilizar bibliotecas de terceiros.
- Validação de dados: existem bibliotecas que nos auxiliam na validação das requisições recebidas. Isso ajuda a prevenir erros inesperados e garante que os dados estejam corretos antes de processá-los.
- Uso de variáveis de ambiente: armazene informações sensíveis, como strings de conexão de banco de dados ou chaves de API, em variáveis de ambiente. O Next.js suporta .env arquivos para gerenciamento de variáveis de ambiente.

O QUE VOCÊ VIU NESTA AULA?

Você aprendeu como funcionam API Routes em projetos Next.js, explorando sua criação, boas práticas e vantagens. Inicialmente, criamos uma requisição HTTP que não utiliza API Routes para posteriormente realizarmos a implementação e notarmos a facilidade no uso, pois foi possível implementar em nosso fluxo sem necessitarmos de configurações adicionais.

EMANDA

REFERÊNCIAS

NEXT.JS. **Documentação Oficial**. [s.d.]. Disponível em: <https://nextjs.org/docs/>. Acesso em: 28 jun. 2024.

NIKOLOV, L. **Creating and using API Routes in Next.JS**. 2023. Disponível em: <https://medium.com/@NikolovLazar/creating-and-using-api-routes-in-next-js-7c9798e67684>. Acesso em: 28 jun. 2024.

VERCEL. **Vercel**. [s.d.]. Disponível em: <https://vercel.com/blog/>. Acesso em: 28 jun. 2024.

PALAVRAS-CHAVE

NextJs. ApiRoutes. HTTP Request.

EMENDAS

POS TECH